Take-Home Assignment

Automation & AI Workflow Associate

Company X

Start date: Tuesday, 2 June 2026

Deadline: Sunday, 7 June 2026 — 11:59 PM IST

Tech stack you'll touch: Python (backend) · Next.js (frontend) · n8n / Zapier / Make (automation) · An LLM (Claude preferred; OpenAI or any other is fine)

Submission: Reply to the email you received this from, with a GitHub link (or zip) and a short Loom (2–3 mins)

Why this assignment

This role is about gluing things together — pulling data from one place, processing it with code or an LLM, and pushing it into a tool that someone non-technical can actually use. The assignment mirrors a real piece of work you'd do in the first month here. We're not looking for polish. We're looking for working end-to-end flow, clean code, and good judgment about when to use no-code vs Python.

The scenario

Company X gets inbound leads through multiple channels — a website form, WhatsApp, and email. Today, someone manually copies each lead into a spreadsheet, reads the message, decides if it's worth replying to, and then either ignores it or sends a quick reply. We want this automated.

You'll build a small system that does the following:

- Accepts a new lead (from a CSV file, a Google Sheet, or an API call — your choice)
- Uses an LLM to classify the lead as Hot, Warm, or Cold based on the message text
- Uses the LLM to draft a 1–2 sentence personalized reply
- Stores the result somewhere queryable
- Shows the leads in a simple dashboard that a non-technical user could operate

What to build

Part 1 — Python backend (required)

Build a small FastAPI (or Flask) service that exposes at minimum:

- POST /lead — accepts {name, email, phone, message, source} and stores it

- GET /leads — returns all leads with their classification and suggested reply
- POST /classify — takes a message string, returns {classification: 'Hot'|'Warm'|'Cold', suggested_reply: string}

The /classify endpoint should call an LLM (Claude preferred via the Anthropic API; OpenAI is fine; even a local model is fine). Storage can be SQLite, a JSON file, or even an in-memory dict — we don't care which, as long as GET /leads returns what was POSTed.

We want to see clean code: meaningful variable names, a couple of comments, and basic error handling around the LLM call.

Part 2 — Automation workflow (required)

Build one of these in n8n, Zapier, or Make.com:

- A flow that triggers when a new row is added to a Google Sheet (or a new Google Form is submitted), sends the lead to your /lead endpoint, and posts a notification to Slack, Telegram, email — anywhere visible
- OR a flow that runs on a schedule (every hour), pulls all 'Hot' leads from your /leads endpoint, and posts a digest somewhere

Export the flow as JSON (n8n exports easily) or share screenshots showing the steps. If you don't have a paid Zapier/Make plan, use n8n locally (it's free and open source).

Part 3 — Next.js frontend (required)

A small Next.js app (App Router or Pages Router, your choice) with:

- A homepage that fetches GET /leads from your Python backend and renders them in a table
- Columns: Name · Email · Phone · Source · Message (truncated) · Classification (color-coded) · Suggested Reply
- A filter dropdown to show only Hot / Warm / Cold leads
- A 'Mark as Contacted' button per row that updates the backend (you can add a status field for this)

Use Tailwind or plain CSS. We don't expect a designed UI — we expect a clean, functional one. If you're more comfortable with React (Create React App / Vite) than Next.js, that's acceptable — but Next.js is preferred because that's what we use.

Part 4 — README (required)

A single README.md that includes:

- How to run each part locally (one command per part if possible)
- A short architecture sketch — even ASCII art is fine — showing how the three parts connect
- What you'd improve with more time, and what tradeoffs you made
- Any libraries / APIs / tools you used and why

Part 5 — Demo Loom (required)

A 2–3 minute screen recording that shows:

- Your automation flow triggering

- The Python backend receiving the lead and classifying it
- The Next.js page showing the new lead with its classification
- One click of the 'Mark as Contacted' button

Sample lead data

We've attached a CSV (sample_leads.csv) with 10 fake leads. Use these as your test data — same input for every candidate means we can compare submissions fairly. You're welcome to add more leads if you want to stress-test your flow.

Ground rules

- Don't deploy to production — run everything locally.
- Don't write a test suite unless you want to. We won't grade tests.
- Don't build authentication, accounts, or a database with migrations. A JSON file or SQLite is enough.
- DO use AI assistants (Cursor / Copilot / Claude / ChatGPT) — we'd expect you to in real work. Just be ready to explain every line of code in the interview.
- DON'T fully outsource the assignment to AI. The Loom + interview will catch this fast.

How to submit

Reply to the email this came from, before the deadline, with:

- A link to a public GitHub repo (or a zip file)
- A link to your Loom (or any screen recording — Google Drive video, YouTube unlisted, etc.)
- Your README must be inside the repo at the top level

Questions?

If anything in this brief is unclear, just reply to the email and ask. We'd much rather you ask early than guess wrong. Asking smart questions is part of what we're hiring for.

Good luck — looking forward to seeing what you build.